Тема 12: Процедурно програмиране - указатели, масиви и рекурсия

Анотация:

- Указатели и указателна аритметика.
- Едномерни и многомерни масиви.
    - Основни операции с масиви - индексиране.
    - Сортиране и търсене в едномерен масив - основни алгоритми.
- Рекурсия
    - пряка и косвена рекурсия, линейна и разклонена рекурсия.
- Символни низове.
    - Представяне в паметта.
    - Основни операции със символни низове

Въведение

Процедурното програмиране е парадигма, при която програмата се организира като последователност от инструкции, групирани в процедури (или функции), които обработват данни чрез променливи. В основата на този подход стоят детерминирани стъпки и структуриран контрол на потока, което го прави удобен за решаване на ясно дефинирани задачи.

В процедурното програмиране променливите играят централна роля - те представляват наименувани области в паметта, в които се съхраняват стойности от определен тип (цяло число, символ, логическа стойност и др.). Всяка променлива е разположена на конкретно място в паметта, което се нарича адрес. Адресът е просто цяло число, което указва позицията на съответната клетка в адресното пространство на компютъра. Разбирането на адресите е основа за работата с указатели - променливи, които не съхраняват стойности директно, а адреси на други променливи.

Разбирането на типовете данни и тяхното представяне в паметта е ключово за ефективното използване на указатели, масиви и рекурсия - основни инструменти за изграждане на по-сложни алгоритми и структури от данни.

Компютърната памет е основният ресурс, в който програмите съхраняват данни по време на изпълнение. Паметта може да бъде статична (заета по време на компилиране) или динамична (заета по време на изпълнение). Управлението на динамичната памет чрез указатели позволява ефективна работа с променливи размери на данните, като масиви и структури, които не са известни предварително при компилация. Това е особено важно при създаване на сложни структури от данни и оптимизиране на използването на ресурси.

Тази тема разглежда три важни концепции:

- Указателите - механизъм за директен достъп до паметта и динамично управление на данни;

- Масивите - структури от множество еднотипни стойности, достъпвани чрез индекси;
- Рекурсията - техника, при която функция извиква сама себе си, използвана при алгоритми с повтаряща се или вложена структура.

Указатели

Дефиниция и синтаксис

Указател (pointer) е променлива, която съхранява адреса в паметта на друга променлива от определен тип. Паметта в един компютър се състои от поредица байтове, които могат да бъдат индексирани чрез цели числа (адреси). Указателят "помни" адреса на дадена променлива.

```
<тип> *име;
```

- Указателят е с фиксиран размер според машината.
    - Ако машината е 32-битова, указателя заема 4 байта.
    - Ако машината е 64 битова, указателя заема 8 байта от паметта.
- Извличане на адрес и инициализация на указател
    - Адресът на дадена променлива се извлича с помощта на унарния оператор &, който се прилага към нея.
    - Пример: ако имаме променлива int num = 489;, искаме да вземем нейния адрес с &num.
    - Унарният оператор & може да се прилага само върху lvalue данни — тоест такива, които имат адрес в паметта.
    - Не може да се прилага върху rvalue данни — стойности, които нямат адрес (напр. временни изрази).
    - Инициализация на указател става чрез задаване на адреса на променливата:

```
int num = 42;
int* ptr = &num;
```

- типът на указателя трябва да съвпада с типа на данната, към която сочи.

```
// Важно: типът на указателя трябва да съвпада с типа на променливата, която
сочи.
float pi = 3.14;
float* pi_ptr = &pi;     // правилно
int* wrong_ptr = &pi;    // грешка: различни типове
```

- Операторът * се използва за достъп до стойността, намираща се на даден адрес (дереференция).

```
int num = 10;
int* ptr = &num;    // ptr сочи към адреса на num
std::cout << *ptr;  // отпечатва 10
std::cout << ptr;   // отпечатва адресът на num
std::cout << &ptr;  // отпечатва адресът на ptr
```

Видове указатели

- Указател към указател

```
int a = 5;
int* p = &a;
int** pp = &p;

std::cout << **pp; // 5
```

- Нулев указател (nullptr)
    - Pointer literal със специална стойност за нулев адрес (адрес 0x00000000).
    - Използва се за по-безопасно и ясно указване на „нулиран“ указател.
    - Ползваме го за стойност по подразбиране на неинициализиран указател.

```
  int* ptr = nullptr
```

- const указатели

```
// Указател към константа
const int* p1 = &x; // не можем да променяме *p1

// Константен указател
int* const p2 = &x; // не можем да променим p2

// Константен указател към константа
const int* const p3 = &x;
```

Основни операции с указатели

Указателите могат да участват в:

- аритметика
    - При аритметика, стойността на указателя се увеличава/намалява със стойността на sizeof(тип).

```
  int* p;     // +1 -> +4 байта
  double* q;  // +1 -> +8 байта
```

        - Формално, ако `p` е указател от тип `T`, то инкрементирането на `T*` с `i` е еквивалентно на `p + i * sizeof T`, където `sizeof T` връща броя байтове, необходими за съхраняване на данна от тип `T`

- логически сравнения
    - При прилагането на оператори за сравнение към указатели се сравняват стойностите на адресите, които те съдържат.
    - Два указателя са равни, ако сочат към един и същ адрес (т.е. към една и съща променлива).

```
int x = 5, y = 10;
int* px = &x;
int* py = &y;

if (px != py)
    std::cout << "Указателите сочат към различни променливи";
```

Потенциални грешки при работа с указатели

- Uninitialized pointer - указател с произволна стойност (не е инициализиран);
- Dangling pointer - указател към освободена памет;
- Memory leak - при динамична памет без delete;
- Dereferencing nullptr - достъп до нулев указател води до runtime грешка.

Масиви

Масивът е съставен тип, който представлява линейна структура от данни – множество стойности от един и същи тип, разположени последователно в паметта. Всеки елемент се достъпва чрез индекс, като индексите започват от 0.

```
<тип> <име>[брой_елементи];
```

- Едномерни масиви
    - Деклариране:

```
int arr[5] = {1, 2, 3, 4, 5};
```

    - Достъп:

```
int x = arr[2];  // x = 3
```

- Двумерни масиви
    - Двумерният масив е логическа структура от редове и колони, която се представя в паметта като последователен (линеен) блок от данни.
    - Всеки елемент се достъпва чрез два индекса - един за ред и един за колона, но реално се намира на изчислен адрес.
        - Деклариране:

```
int matrix[3][4];
matrix[1][2] = 7; // достъп до елемент на 2-ри ред и 3-та колона

// Визуализация
Редове       [0]       [1]       [2]
   ----------------------------
Колони      [0][0]   [1][0]   [2][0]
            [0][1]   [1][1]   [2][1]
            [0][2]   [1][2]   [2][2]
            [0][3]   [1][3]   [2][3]
```

        - Инициализация чрез цикъл:

```
for (int i = 0; i < 3; i++)
    for (int j = 0; j < 4; j++)
        matrix[i][j] = i + j;
```

Указатели и масиви

Името на масив е указател към първия му елемент:

```
int arr[] = {1, 2, 3};
int* p = arr;
```

```
std::cout << *(p + 1); // 2
```

- Чрез указатели можем да обхождаме масиви;
- arr[i] е еквивалентно на *(arr + i).

Указатели и функции

Указателите позволяват предаване по адрес (по референция):

```
void update(int* p) {
    *p = 100;
}

int a = 5;
update(&a); // a става 100
```

Масиви като формални параметри

- Едномерни масиви
    - Когато подаваме масив на функция, той не се копира, а се предава по указател.
    - Това означава, че функцията работи директно с оригиналния масив и всички промени в него ще се отразят и извън функцията.

```
  void printArray(int arr[], int size);
```

    - Следните форми са еквивалентни и се интерпретират като int* arr:

```
  void func(int arr[]);
  void func(int arr[10]);
  void func(int* arr);
```

    - Това позволява оптимизация - не се създава локално копие на масива, което би отнело излишна памет и време.
- Многомерни масиви
    - При многомерни масиви също се предава указател, но трябва да се уточнят размерите на всички измерения, с изключение на първото.

```
  void printMatrix(int arr[][4], int rows);
```

    - В този пример не задаваме броя редове (първото измерение), но трябва задължително да уточним броя колони, за да може компилаторът да изчисли коректно адресите на елементите.
    - Важно: Това правило важи и за масиви с повече от две измерения. Размерите след първото винаги трябва да бъдат уточнени.

Динамични масиви

Освен статичните масиви, съществуват и динамично заделени масиви, които се разполагат в различна част от паметта — т.нар. heap.

Секция	**Описание**
Код на програмата	Съдържа машинните инструкции, които се изпълняват
Глобални/статични данни	Памет за глобални и статични променливи
Stack (стек)	Използва се за извиквания на функции, аргументи и локални променливи
Heap (купа)	Динамично заделяна памет по време на изпълнение

Динамичната памет в heap-а се заделя чрез оператора new. Той връща указател към първия елемент на заделената памет.

```
int* arr = new int[10];
```

- Тази команда заделя 40 байта (10 елемента по 4 байта) в heap.
- arr е указател към първия елемент от този динамичен масив.

Паметта, заделена с new, не се освобождава автоматично. Задължение на програмиста е да я освободи ръчно, когато вече не е необходима, чрез оператора delete.

```
delete ptr; // за единична стойност
delete[] arr; // за масив

// След освобождаване на паметта е добра практика указателят да бъде присвоен
към nullptr, за да не сочи към освободена (и потенциално опасна) памет.
ptr = nullptr;
arr = nullptr;
```

При динамично заделяне на двумерен масив в heap паметта не можем да използваме new T[10][10], тъй като това води до едно цяло блоково заделяне, което не е гъвкаво за освобождаване. Вместо това заделяме поетапно:

```
int** matrix = new int*[10]; // заделяме масив от 10 указателя (редове)

for (int i = 0; i < 10; ++i) {
    matrix[i] = new int[10]; // заделяме по 10 елемента за всеки ред
}
```

Освобождаване на динамичен двумерен масив - Паметта трябва да се освобождава ред по ред, след което се изтрива и основният масив от указатели:

```
    delete[] matrix[i]; // освобождаваме всеки ред
}
delete[] matrix; // освобождаваме масива от указатели
matrix = nullptr; // добра практика
```

Така се избягват memory leaks - ситуации, при които заделена памет не се освобождава и остава неизползваема, което с времето може да доведе до забавяне или срив на програмата. Чрез правилно освобождаване гарантираме, че всички ресурси са върнати обратно на системата.

Основни операции с масиви - Индексиране

- Достъп до елемент чрез позиция (индекс), напр. arr[3];
- Индексите започват от 0;
- Извъндиапазонен достъп води до неопределено поведение.

Сортиране в едномерен масив - Основни алгоритми

- Bubble Sort
    - Повтарящо сравнение и размяна на съседни елементи.

```
bool swapped;
for (int i = 0; i < n - 1; ++i) {
    swapped = false;
    for (int j = 0; j < n - i - 1; ++j) {
        if (arr[j] > arr[j + 1]) {
            std::swap(arr[j], arr[j + 1]);
            swapped = true;
        }
    }
    if (!swapped)
        break; // Array is already sorted
}
```

    - Подходящ за много малки масиви
    - Той е лесен за имплементиране, но е неефективен за големи данни ($O(n^2)$).
- Selection Sort
    - На всяка итерация намира най-малкия елемент и го поставя в началото

```
for (int i = 0; i < n - 1; ++i) {
    int min_idx = i;
    for (int j = i + 1; j < n; ++j) {
        if (arr[j] < arr[min_idx]) {
            min_idx = j;
        }
    }
    if (min_idx != i) std::swap(arr[i], arr[min_idx]);
}
```

    - Използва се, когато е необходим прост алгоритъм с минимални обменни операции, но също е бавен за големи масиви ($O(n^2)$).
- Insertion Sort
    - Всеки следващ елемент се вмъква в вече сортираната част.

```
for (int i = 1; i < n; ++i) {
    int key = arr[i];
    int j = i - 1;
    while (j >= 0 && arr[j] > key) {
        arr[j + 1] = arr[j];  // shift element right
        --j;                  // move left
    }
    arr[j + 1] = key;
}
```

    - Добър избор за почти сортирани масиви или малки набори от данни, тъй като

работи бързо в такива случаи (близо до O(n)).

Търсене в едномерен масив - Основни алгоритми

- Линейно търсене (Linear Search)
    - Подходящ, когато масивът не е сортиран или е много малък, тъй като просто проверява всеки елемент (O(n)).

```
for (int i = 0; i < n; ++i)
    if (arr[i] == target)
        return i;
return -1;
```

- Двоично търсене (Binary Search)
    - Използва се само върху сортирани масиви.
    - Необходим е предварително сортиран масив, за да функционира правилно.
    - Много ефективен за големи масиви с време на търсене O(log n).

```
int l = 0, r = n - 1;
while (l <= r) {
    int mid = l + (r - l) / 2;
    if (arr[mid] == target) return mid;
    else if (arr[mid] < target) l = mid + 1;
    else r = mid - 1;
}
return -1;
```

Рекурсия

Определение

Рекурсията е техника в програмирането, при която функцията извиква сама себе си с цел решаване на задача чрез разбиване на проблема на по-малки, подобни подпроблеми.

В теорията на изчислимостта е доказано, че всяка рекурсивна функция може да бъде преобразувана в итеративна, но рекурсията често е по-интуитивен и удобен начин за описание на естествено рекурсивни задачи (напр. факториел, Фибоначи, обхождане на дървета).

Видове рекурсия

- Пряка рекурсия
    - Функцията извиква директно сама себе си.

```
void func() {
    func(); // директно извикване
}
```

- Косвена (непряка) рекурсия
    - Една функция извиква друга, която на свой ред извиква първата, създавайки цикъл от извиквания.

```
void A() { B(); }
void B() { A(); }
```

- Линейна рекурсия
    - Функцията извиква себе си точно веднъж във всяко изпълнение.
    - Най-често срещаната форма на рекурсия.

```
int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}
```

- Разклонена рекурсия
    - Функцията извиква себе си повече от веднъж по време на изпълнение.

```
void func(int n) {
    if (n > 0) {
        func(n - 1);
        func(n - 2);
    }
}
```

Етапи при изпълнение на рекурсията

- Разгъване (Expansion)
    - Функцията се извиква многократно, създавайки стекови рамки и разделяйки проблема на подпроблеми.
- Свиване (Collapse)
    - Започва връщането на резултати от най-дълбокото извикване към първоначалното, събирайки отговорите.

Характеристики на рекурсивните функции

- Дълбочина на рекурсията
    - Максималният брой вложени извиквания преди достигане на базовия случай.
- Базов случай (дъно)
    - Условие, при което рекурсивната функция не прави ново извикване, а започва да връща резултати назад
    - Задължителен, за да се избегне безкрайна рекурсия.

Практически съображения

- Рекурсията осигурява елегантност и краткост, но може да е трудна за четене и поддръжка.
- Липсата на правилен базов случай може да доведе до безкрайна рекурсия и срив на програмата (stack overflow).
- Рекурсивните извиквания използват стекова памет, която е ограничена.
- В реални приложения често се предпочитат итеративни решения заради по-голяма ефективност и по-добър контрол върху ресурсите.

Пример за използване на рекурсия с реална задача -Факториел на число (n!)

Факториелът на число n е произведението на всички цели числа от 1 до n. Рекурсивното изчисление е естествено, защото:

```
n! = n * (n-1)!
0! = 1 (базов случай)
```

Решение:

```
int factorial(int n) {
    if (n <= 1)  // базов случай
        return 1;
    else
        return n * factorial(n - 1);  // рекурсивно извикване
}
```

Символни низове

Символните низове представляват последователности от символи, използвани за обработка на текстова информация. В езика C++ те могат да бъдат представени по два основни начина:

- чрез C-низове (низови масиви)
    - Основани на масив от символи
    - Използват се в по-ниско ниво на програмиране
    - Манипулацията с тях изисква използването на функции от библиотеката
    - Необходимо е ръчно управление на паметта при динамични низове
- чрез обекти от класа std::string, дефиниран в стандартната библиотека.
    - Дефинирани в заглавния файл , част от STL (Standard Template Library).
    - Осигуряват удобен и безопасен интерфейс за работа с низове.
    - Позволяват използването на оператори, методи и динамично управление на паметта.

Представяне на символи и низове в паметта

- Типът char в C++ е предназначен за съхраняване на един символ.
- Всеки символ е представен чрез цяло число според ASCII (American Standard Code for Information Interchange).
- Диапазонът на стойностите за char обикновено е [0, 255] при unsigned char или [-128, 127] при signed char, в зависимост от компилатора.
- Размерът на char е 1 байт (8 бита).

```
// Статичен низ чрез класа `string`
string h = "hello";
// Същият низ като масив от символи
char ch[] = {'h', 'e', 'l', 'l', 'o', '\0'};
```

Основни операции със символни низове

Операция	**Функция**	**Описание**
Дължина	`strlen(str)`	Връща броя символи без `'\0'`
Копиране	`strcpy(dst, src)`	Копира съдържанието на `src` в `dst`
Конкатенация	`strcat(dst, src)`	Добавя `src` в края на `dst`
Сравнение	`strcmp(s1, s2)`	Сравнява лексикографски; връща 0, ако са равни
Търсене	`strchr(str, ch)`	Намира първото срещане на символ `ch` в `str`
Извличане на подниз	`strncpy(dst, src, n)`	Копира до `n` символа

Забележка: Масивът представлява последователност от елементи от един и същи тип, подредени в непрекъснат блок от памет. Всеки елемент има индекс, започващ от 0, което позволява достъп чрез квадратни скоби: array[index].